Ringbuffer

A high-performance, cross-platform circular buffer library designed for binary symmetry and deterministic execution.

What It Does

The Ringbuffer library implements a **three-stage pipelined architecture** (Read → Transform → Write) that connects independent threads through a fixed-size array of shared slots. Each slot carries its own state machine, checksum, and data buffer, enabling lock-free concurrent processing with cache-line-aligned counters to prevent false sharing.

How the pipeline works:

1. **Read thread** fills slots from an input file descriptor, marking each slot `readyfortransform` when full. On EOF, it marks the final slot with `LAST_BUFFER_TO_TRANSFORM`.
2. **Transform thread** applies a user-supplied `std::function` to each slot's data, computes an atomic checksum (MD5/SHA256/SHA512), and marks the slot `readyforwrite`.
3. **Write thread** writes transformed data to an output file descriptor, clears the slot buffer, and resets it to `readyforread`. On the last buffer, it flushes (`fsync`) before signaling completion.

Each stage uses its own independent atomic index counter, so threads operate on slots without contention. The `wait()` method blocks until all data has flowed through the entire pipeline and threads have joined cleanly via `std::jthread` cooperative cancellation.

Overview

The Ringbuffer library implements a three-stage pipelined architecture (**Read** → **Transform** → **Write**) to ensure efficient data movement and processing. The core focus of the project is **binary symmetry**, meaning the source code remains identical across all supported operating systems to ensure maintainability and predictable behavior.

Primary Goal

The objective is to provide a prompt-driven, immediate build-and-run functionality where the codebase can be regenerated or updated across multiple platforms using a single set of functional specifications.

Key Specifications

- **Multi-OS Support**: Seamless deployment across various Linux distributions and macOS.

- **Multi-Toolchain Compatibility**: Fully buildable using both **GCC** and **Clang**.
- **C++20 Standard**: Leverages `std::jthread`, cooperative cancellation, and advanced synchronization primitives.

Core Features

- **Pipelined Architecture**: Decoupled read, transform, and write loops for maximum throughput.
- **Performance**:
  - **Cache Alignment**: Buffer slots and counters are aligned to 64-byte boundaries to prevent false sharing.
  - **Small Binaries**: Optimized for dead code elimination (`-ffunction-sections`, `-fdata-sections`).
- **Data Integrity**: Integrated checksumming (MD5, SHA256, SHA512) for every buffer slot.
- **Diagnostics**: Non-Virtual Public Interface (NVI) logging system with per-stage (read/transform/write) independently configurable verbosity levels.

Quick Start: End-to-End Workflow (macOS Example)

The following sequence shows the complete prompt-driven workflow for setting up, building, verifying, and testing on macOS. Each step depends on the previous one — execute them in order.

Step 1: Generate Source Code

```
"Use prompts/macos/cpp/create_functional.md to perform a full generation of the ringbuffer f
```

Produces: `ringbuffer.hpp`, `status.hpp`, `swtr_ringbuffer_impl.hpp`, and all `.cpp` implementation files in `src/macos/cpp/`.

Step 2: Create Build Environment

```
"Use prompts/macos/cpp/create_build.md to create the build environment for macos"
```

Produces: `Makefile`, `version.mk`, and built library artifacts (`.a` static, `.dylib` shared) in `src/macos/cpp/lib/`.

Step 3: Verify the Build

```
"Use prompts/common/verify_build.md to run build_verify for macos"
```

Confirms: library compiles, produces correct Mach-O binaries, symbol visibility is correct, and binary sizes are within expected ranges.

Step 4: Set Up Test Infrastructure

`"Use prompts/macos/cpp/create_regression.md to create the test environment for macos"`

Produces: `tests/` directory with Makefile, test data (T001–T007), utility scripts, and builds `test001_staticlib` / `test001_dynamiclib`.

Step 5: Execute Regression Tests

`"Use prompts/common/run_regression.md to run regression_test on macos"`

Executes: the full T001–T007 test matrix, generates `regression_report.md` and `regression_report_junit.xml`.

Incremental Updates

After the initial setup, use update prompts to synchronize with changed specifications:

`"Use prompts/macos/cpp/update_functional.md to synchronize the macos implementation with upd`

Library API Example

The following example demonstrates the core Ringbuffer API — construction, transform setup, and pipeline execution:

```cpp
#include "ringbuffer.hpp"
#include "swtr_ringbuffer_impl.hpp"
using namespace swtr;

int main() {
    // Transform: convert input to uppercase
    auto transform = [](std::vector<uint8_t>& data, size_t& size) {
        for (size_t i = 0; i < size; ++i)
            if (data[i] >= 'a' && data[i] <= 'z') data[i] -= 32;
    };

    // Create a ring buffer with 4 slots, 4096 bytes each
    auto rb = std::make_unique<SwtrRingBuffer>(4, BufferStatus::readyforread, 4096, nullptr,

    // Configure I/O: read from stdin, write to stdout
    rb->set_read_fd(STDIN_FILENO);
    rb->set_write_fd(STDOUT_FILENO);

    // Enable logging for diagnostics
    rb->set_logging_enabled(true);
    rb->set_output_stream(std::cerr);

    // Run the pipeline: reads until EOF, transforms each slot, writes output
```

```
    rb->start();
    rb->wait();    // blocks until all data flows through read → transform → write
    rb->stop();
}
```

Build with: `clang++ -std=c++20 main.cpp -I../src/macos/cpp -L../src/macos/cpp/lib -lswtr_ringbuffer -o main`

Pipeline Architecture Diagram

The following diagram shows how the three pipeline threads interact through shared slots:

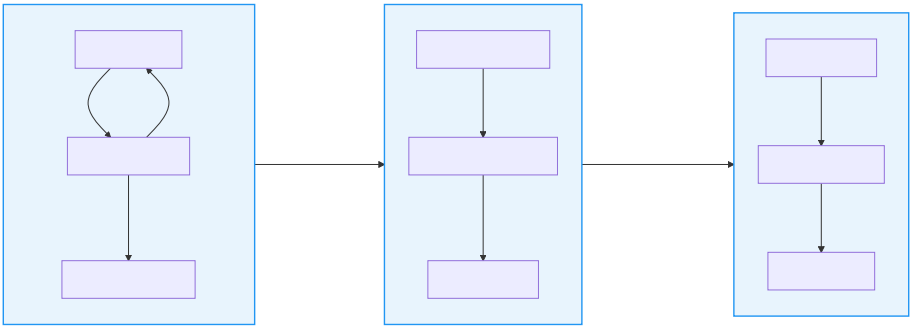


Figure 1: diagram

Key points: - Each stage uses its own independent atomic index counter (`read_index_`, `transform_index_`, `write_index_`) - Slots transition through states: `readyforread` → `processing_read` → `readyfortransform` → `processing_transform` → `readyforwrite` → `processing_write` → `readyforread` - On EOF, the read thread marks the final slot with `LAST_BUFFER_TO_TRANSFORM`, which propagates through transform (`LAST_BUFFER_TO_WRITE`) to write, then all threads break and `wait()` returns

Initial Development Environment

- **Operating System**: macOS 26.5.2 (Darwin 25.5.0)
- **Hardware**: Apple M4 Pro (arm64)
- **AI Tools**:
  - **Agent Harness**: LM Studio Bionic
  - **Model Name**: qwen3.6-35b-a3b-mlx
  - **Model Specs**: 37.75 GB, 162816 context window, parallelism factor 4, local execution.

Project Structure

The project uses a **Deterministic File Mapping** strategy. Logic groups are mapped to specific filenames (e.g., `ReadOps.cpp`, `Threading.cpp`) with a strict constraint of **maximum 3 functions per file** to minimize git collisions and maximize modularity.

Developer & Maintainer Guide

The Golden Rules (Coding Standards)

- **Binary Symmetry**: Source code MUST be identical across all platforms.
- **File Distribution**: No `.cpp` file may contain more than 3 function definitions.
- **Include Discipline**: `ringbuffer.hpp` must be the first non-comment include, followed by alphabetical standard headers, then local headers.
- **Naming**: All private member variables MUST use the trailing underscore convention (e.g., `member_variable_`).
- **Hierarchy**: The implementation must strictly follow the `RingBuffer` → `SwtrRingBuffer` hierarchy.

The Prompt-Driven Workflow

This project is managed via a set of prompts that define the desired state of the codebase.

1. Prompt-Driven Incremental Regeneration The codebase supports incremental synchronization using a state-tracking system (`.gen_state.json`). This allows the AI to update only the files affected by a specification change. - **State Tracking**: Uses SHA-256 hashes of prompts and file fingerprints to detect changes. - **3-Phase Sync**: Execution follows a strict pipeline: **Functional** → **Build** → **Regression**. - **Delta Updates**: "Update" prompts are used to apply targeted diffs, preserving manual overrides and reducing churn.

2. Execution Examples Full Generation (Initial Setup): Use this when adding a new platform or resetting the codebase. - *Prompt:* `prompts/<os>/cpp/create_functional.md` - *Example:* `"Use prompts/ubuntu/cpp/create_functional.md to perform a full generation of the ringbuffer for ubuntu"`

Incremental Update (Partial Regeneration): Use these to synchronize the codebase with updated specifications. - *Functional Update:* `prompts/<os>/cpp/update_functional.md` - *Build Update:* `prompts/<os>/cpp/update_build.md` - *Testing Update:* `prompts/<os>/cpp/update_testing.md` - *Example:* `"Use prompts/ubuntu/cpp/update_functional.md to synchronize the ubuntu implementation with the updated common specifications"`

Verification & Testing: - *Build Verify:* `prompts/common/verify_build.md` - *Regression Test:* `prompts/common/regression_test.md` - *Example:*

`"Use prompts/common/regression_test.md to do regression_test on debian, fedora, macos"`

3. Test Data Pattern System The Ringbuffer project uses a **pattern-driven** approach to test data generation. Patterns are defined in standalone specification files that serve as the single source of truth, ensuring binary symmetry across all platforms and languages.

- **Pattern files** live in `prompts/common/testing/patterns/`. Each file defines:
  - The template string (48 characters)
  - Line format (prefix, payload, suffix structure)
  - The transformation algorithm (e.g., zig-zag space position)
  - A Python reference implementation (ground truth for all shell variants)
  - Validation regex, test file line counts, and JUnit test matrix
- **The `create_scripts.md` prompt** reads a pattern file and generates utility scripts in four shell languages (Bash, Zsh, Tcsh, Bourne) that produce byte-identical output.
- **Scripts are deployed** to each platform's `tests/utils/bin/` directory during test environment setup.

Directory Structure:

```
prompts/common/testing/
  create_scripts.md          ← workflow prompt for generating scripts
  patterns/
      README.md              ← instructions + copy-paste template for new patterns
      zigzag.md              ← the default zig-zag pattern (also the only one)
```

Usage Examples:

- **Generate scripts from the default pattern:**
  - *Prompt:* `prompts/common/testing/create_scripts.md`
  - *Syntax Example:* `"Use prompts/common/testing/create_scripts.md to create utility scripts from the zigzag pattern"`
- **Update scripts after modifying a pattern:**
  - *Prompt:* `prompts/common/testing/create_scripts.md`
  - *Syntax Example:* `"Use prompts/common/testing/create_scripts.md to update the utility scripts. The pattern file is at prompts/common/testing/patterns/zigzag.md"`
- **Create a new pattern and generate scripts:**
  - *Syntax Example:* `"Create a new test data pattern at prompts/common/testing/patterns/sequential.md with template '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ!@#$%^&*()<>', sequential numbering, and validation regex '^[0-9]{4}.{47} [0-9]{4}$'. Then generate the utility scripts."`
- **Verify generated scripts match the Python reference:**

- *Syntax Example:* `"Use prompts/common/testing/create_scripts.md to regenerate scripts from zigzag.md and verify all shell variants produce byte-identical output against the Python reference implementation"`

Supported Platforms

OS	Language	Version
macOS	C++	v1.0.0
Debian	C++	v1.0.0
Fedora	C++	v1.0.0
Ubuntu	C++	v1.0.0
ArchLinux	C++	TBS
FreeBSD 14	C++	TBS
FreeBSD 15	C++	TBS

`*TBS: To Be Supplied | WIP: Work In Progress`

License

This project is licensed under the **BSD 3-Clause License**.